

LangGraph 工作坊

寫出你的第一個 Data Analyst Agent

講師:Levi

github.com/AluminiumShark/langgraph-harness-workshop

 **Open in Colab**

按空白鍵開始 →

今天結束你會看到這個

你不需要寫這個 agent——只要看講師跑、看它怎麼決策。focus 在「它做了什麼選擇」上。

你: 分析 sales.csv, 找出表現最差的區域

agent: [呼叫 inspect_data] [呼叫 run_pandas]

→ 是 East 區域, 營收 \$X, 比平均低 30%

你: 那 West 區呢?

agent: [記得剛才的 csv, 繼續分析]

→ West 區營收 \$Y, 是表現最好的

注意第一輪: agent 自己決定先 inspect_data、再 run_pandas——你沒告訴它要這樣做

注意第二輪: 它記得「剛才的 csv」——多輪對話有記憶

「自己決定」是重點

這才叫 agent,
不是 chatbot

為什麼選 Data Analyst Agent

這個場景剛好同時涵蓋 agent 的三個核心能力——學會它,其他 agent 範例你都看得懂。



對話

MessagesState

多輪互動、記得上一輪、能接續追問
——agent 不是一問一答的 chatbot



工具呼叫

@tool、ToolNode

LLM 不只回文字,它會「叫你」執行某個 Python function——這是 agent 跟世界互動的方式



決策迴圈

LLM 決定下一步

工具跑完,LLM 看結果再決定:再呼叫一次工具、還是回最終答案?這個迴圈是 agent 的本質

學會這個場景 → **90% 的 agent 應用都看得懂**

RAG agent、coding agent、客服 agent...都是這三件事的不同組合

§1

環境救火

拿 Google AI API key

- 1 打開 Colab 連結 → 「Copy to Drive」

 Open in Colab

- 2 到 [Google AI Studio](#) 點 Create API key

- 3 Colab 左側鑰匙圖示 → Add new secret

名稱: `GOOGLE_API_KEY` / 值: 剛才的 key

 打開「Notebook access」開關(常見漏點)

- 4 跑 Cell 1 確認接通

Google AI 的優勢

為什麼今天用 Gemini 不用 OpenAI / Claude?——降低 onboarding 門檻是主因。

✓ 不用刷信用卡就能拿 key

OpenAI / Anthropic 都要綁卡 → 工作坊 5 分鐘卡關率高

✓ 不用建 GCP project

Google AI Studio 跟 GCP 是分開的服務,不需要會 GCP

✓ 不用等審核

點「Create」立即拿 key,免費額度足夠完整堂課

三分鐘搞定

今天用

gemini-2.5-flash

- Google 的**快速**模型(對應 GPT-4o-mini)
- 延遲低、費用是 pro 系列的 1/10
- Tool calling 能力對工作坊綽綽有餘
- **Production** 大量用的也是 **flash** 系列——不是 demo 玩具

學會今天的東西,直接換成 ChatOpenAI / ChatAnthropic 也跑得起來——LangChain 統一介面

第一個 cell

這 cell 做四件事:裝套件 → 載入 API key → 建 LLM 物件 → 試打一句確認接通。

```
!pip install -q -U langgraph langchain langchain-google-genai pandas

import os
from google.colab import userdata
os.environ["GOOGLE_API_KEY"] = userdata.get("GOOGLE_API_KEY")

from langchain_google_genai import ChatGoogleGenerativeAI
llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0.7)
print(llm.invoke("say hi in 5 words").text())
# → 'Hi there, how are you?'
```

line 1: `!pip install` 是 Colab/Jupyter 語法,在 cell 裡跑 shell 指令

line 3-5: 從 Colab Secrets 載入 key 到環境變數——LangChain 預設讀 `GOOGLE_API_KEY`

line 7-8: 建一個 LLM 物件——還沒呼叫 API

line 9: 真的呼叫 API——這行會花錢/扣免費額度

看到 **Gemini** 回了一句話 = 環境就緒

§2

LangGraph 心智模型

LangGraph 把 AI 流程畫成一張
圖

LangGraph 的全部

LangGraph 看似複雜,核心就**這三個概念**。今天前半場全部都在玩這三個東西。

節點 Node

一個處理步驟 — 本質是個 Python function

例如「檢查資料」「呼叫 LLM」「執行工具」

邊 Edge

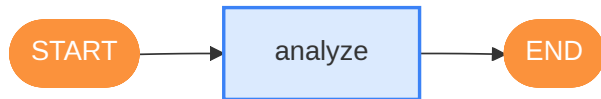
步驟之間的流向 — 從節點 A 接到節點 B

後面會看到「條件邊」:依 state 動態決定走哪

狀態 State

節點之間傳遞的資料包 — 一個 dict

節點不直接互相呼叫,只透過 state 傳資料



這就是 LangGraph 的全部

§3

Python 速補

你會看到的四個符號

這節 30 分鐘投資、後面 \$4 \$7 \$8 都受惠。學員不寫 code,但要看得懂講師螢幕上每個符號是什麼意思——這四個是接下來會反覆出現的。

```
x: int
```

Type hint

標註變數型別 — 到處都是的基本語法

```
class State(TypedDict)
```

TypedDict

dict 的設計圖 — LangGraph 的 State 都用這個

```
Annotated[list, ...]
```

Annotated

給型別夾帶提示 — MessagesState 用它告訴 LangGraph 怎麼合併訊息

```
@tool
```

裝飾器

把 function 包成 LLM 看得到的工具 — Tool calling 的入口

重點不是會寫,而是看得懂

§3.1 Type hint

Python 的「給變數加標籤」語法。重點:那個冒號 `:` 是型別標籤,不是定義變數的語法。

變數: 型別 = 值

```
# 基礎變數
age = 20

# 加上 type hint
age: int = 20

# Function 也可以
def greet(name: str) -> str:
    return f"Hello, {name}"
```

語法

冒號後寫型別,等號後寫值。讀作「age 是一個 int,值是 20」

不強制

Python 不會檢查。 `age: int = "hello"` 也跑得起來,不報錯

給人看

提示讀者跟 IDE。LangGraph 框架也會讀這個資訊

看到 `xxx: yyy` 都讀成「xxx 是一個 yyy 型別」

§3.2 TypedDict — dict 的設計圖

TypedDict 看起來像 class,但它定義的不是 **class**,是「**dict** 的 **schema**」。LangGraph 規定 State 要用這個格式,所以非懂不可。

```
from typing_extensions import TypedDict

# 設計圖:User 應該有 name 和 age
class User(TypedDict):
    name: str
    age: int

# 實際使用 → 普通的 dict
alice: User = {"name": "Alice", "age": 20}
print(alice["name"]) # "Alice"
```

typing_extensions 是什麼?

Python 內建 typing 套件的進階版,記住寫法即可,不用深究

為什麼用 class 關鍵字?

語法借用而已,實際產出是 dict 的型別標籤,沒有 self、沒有 method

底層仍然是 **dict** — 存取一定用 `["key"]`,不是 `.attr` ← 學員最常踩的雷

Dict vs Object 的差別

這兩種「資料容器」在 Python 裡是**完全不同的東西**——沒寫過 class 的人最容易搞混。

Dict 風格:中括號 + 字串

```
my_dict = {"name": "Alice"}

my_dict["name"]    # ✓ "Alice"
my_dict.name       # ✗ AttributeError
```

Object 風格:點

```
class User:
    def __init__(self):
        self.name = "Alice"

u = User()
u.name           # ✓ "Alice"
u["name"]        # ✗ TypeError
```

TypedDict 雖然用 `class` 關鍵字定義,底層仍是 **dict**

所以一定用 `d["key"]` 不是 `d.attr` — 學員最常踩的雷

§3.3 Annotated — 給型別夾帶提示

Annotated 讓你給型別夾帶額外資訊。Python 本身不會用這資訊,但框架(例如 **LangGraph**)可以讀——這是它存在的意義。

```
from typing_extensions import Annotated
```

```
x: Annotated[int, "這是一個年齡"]
```

`int`

真正的型別

"這是一個年齡"

夾帶的附加資訊

§7 會看到的關鍵用法:

```
messages: Annotated[list, add_messages]
```

```
#
```

```
↑
```

```
↑
```

```
# 這是個 list
```

```
用 add_messages 合併
```

```
#
```

```
不要直接覆蓋
```

§3.4 裝飾器 — @xxx 是什麼

裝飾器聽起來很玄,其實就是「把 **function** 包一層」的語法糖。今天唯一會用到的裝飾器是 `@tool`。

```
def get_weather(city):  
    return f"{city} 今天晴天"  
  
# 加上 @tool 之後  
from langchain_core.tools import tool  
  
@tool  
def get_weather(city: str) -> str:  
    """查詢城市的天氣。"""  
    return f"{city} 今天晴天"
```

@tool 等於這行:

```
get_weather = tool(get_weather)
```

意思是:把原本的 `get_weather` **function** 餵給 `tool` 處理一下,然後把處理結果重新存回 `get_weather` 這個變數。新版本的 `get_weather` 多了「告訴 LLM 我長怎樣」的能力。

裝飾器的 mental model

```
@xxx  
def foo(...):  
    ...
```



等於

```
def foo(...):  
    ...  
foo = xxx(foo)    # 拿 foo 餵給 xxx, 把回傳值重新叫 foo
```

寫在 function 上面那行,以 @ 開頭

本質是**把 function 包一層**

§3 小結

符號	名稱	作用	等下會在哪看到
<code>x: int</code>	type hint	標註型別	到處
<code>class State(TypedDict)</code>	TypedDict	dict schema	§4 echo bot
<code>Annotated[list, add_messages]</code>	Annotated	夾帶提示	§7 MessagesState
<code>@tool</code>	裝飾器	包成工具	§8 Tool Calling

看不懂隨時打斷我問 → 進 §4 開始寫第一個 graph

§4

第一個 Graph
echo bot

最小可跑的 graph

LangGraph 工作流的四個必要元素。學會這四個,你就能寫出任何 graph——後面的 tool、routing、checkpoint 都是這四個的擴充。



State

傳什麼資料

用 TypedDict 定義 schema



Node

state → dict

普通 function, 吃 state 回 dict



compile

變成可跑的

把圖定義轉成可執行物件



invoke

餵 state、拿結果

給初始 state, 跑完拿 final state

不教 tool、不教 routing — 只看這四件事

完整 echo bot

讀的順序: 定義 State schema → 寫 Node function → 組裝 graph → compile → invoke。

```
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
    user_input: str
    response: str

def echo(state: State) -> dict:
    return {"response": f"你說的是:{state['user_input']}"}

builder = StateGraph(State)
builder.add_node("echo", echo)
builder.add_edge(START, "echo")
builder.add_edge("echo", END)

graph = builder.compile()
result = graph.invoke({"user_input": "你好"})
print(result)
# {'user_input': '你好', 'response': '你說的是:你好'}
```

Node 的關鍵概念

理解了「merge」就理解了 Node。你的 **function** 不需要回完整 **state**, 只回要更新的欄位——LangGraph 自動幫你 merge。

return 的 dict 會被自動 **merge** 進 state

進 echo 之前

```
{"user_input": "你好",  
  "response": ""}
```

echo return 的

```
{"response":  
  "你說的是: 你好"}
```

merge 後

```
{"user_input": "你好",  
  "response":  
    "你說的是: 你好"}
```

你只 return 要更新的欄位就好 — 不用每次把整個 state 抄一遍

這個設計讓 graph 越大越方便: 你不用知道其他 node 在做什麼, 只管自己負責的部分

動腦不動手:猜輸出

```
def echo(state: State) -> dict:  
    return {"response": state["user_input"].upper() }
```

輸入 "hello world"

猜猜 **print(result)** 印出什麼?

§5

兩節點 + 邊

兩節點透過 state 溝通

這是 \$4 echo bot 的延伸——多一個節點。注意 **clean** 跟 **respond** 之間沒有直接呼叫,只透過 **state["cleaned"]** 傳資料。

```
class State(TypedDict):
    user_input: str
    cleaned: str
    response: str

def clean(state: State) -> dict:
    return {"cleaned": state["user_input"].strip().lower()}

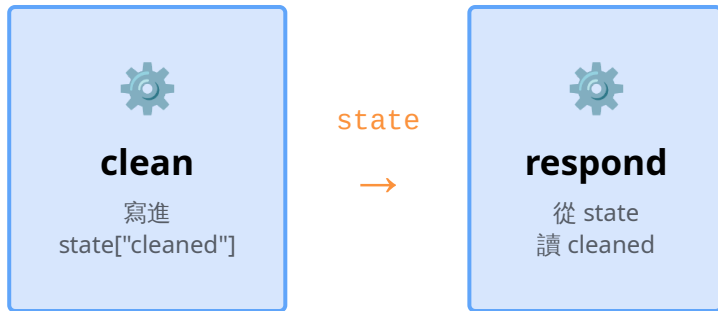
def respond(state: State) -> dict:
    return {"response": f"處理後:{state['cleaned']}"}

builder = StateGraph(State)
builder.add_node("clean", clean)
builder.add_node("respond", respond)
builder.add_edge(START, "clean")
builder.add_edge("clean", "respond")
builder.add_edge("respond", END)
```

跟 echo bot 結構一樣,只是多加了一個節點 + 一條邊

水管比喻

理解這個比喻 → 理解 LangGraph 的所有設計選擇。你蓋了 **graph**,其實是在鋪「**state** 流經哪些節點」的水管。



兩節點之間 **完全不直接互相呼叫**

state 是節點之間唯一的水管

好處:節點之間解耦——你能單獨換掉 respond 不影響 clean

代價:欄位要顯式寫進 State schema——多打幾行字

§6

LLM 物件解剖 (打開黑箱)

為什麼這節是關鍵節

學員不寫 code, LLM 回什麼物件、物件長怎樣

只能靠講師現場 **print** 給他們看

這節投資紮實, 後面 §7 §8 §11 §12 都會省下時間

§6.1 LLM 怎麼呼叫

這是 LangChain 呼叫 LLM 的標準四步:import → 建 LLM 物件 → 餵 messages list → 拿回 AIMessage。

```
from langchain_google_genai import ChatGoogleGenerativeAI

llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0.7)

response = llm.invoke([
    {"role": "user", "content": "你好"}
])

print(type(response))
# <class 'langchain_core.messages.ai.AIMessage'>

print(response.text())
# '你好!有什麼我可以幫你的嗎?'
```

注意兩件事:

- ① `llm.invoke()` 吃的不是「字串」而是「list of dicts」——要長這樣才符合對話格式
- ② 拿訊息文字用 `.text()` (這是個 method 不是 attribute,記得加括號)——下兩頁解釋為什麼不用 `.content`

重點解剖

四個重點都要記得——這是 **LLM API** 的基本協議,後面 tool calling、MessagesState 都基於這個結構。

① `llm.invoke(messages)` 吃一個 **list of dicts**

② 每個 dict 有 `"role" ( user / assistant / system )+ "content"`

user 是使用者、assistant 是 LLM、system 是給 LLM 的指令

③ 回傳是 `AIMessage` 物件,不是字串!

要拿文字內容用 `.text()` ——這是 method 不是 attribute(下頁解釋為什麼不用 `.content`)

④ `AIMessage` 不只 Gemini,Claude / OpenAI / Bedrock 都用同一個類別

→ LangChain 的價值之一:統一不同 LLM 廠商的回傳格式

把 AIMessage 整個打開看

這是整個前半場最重要的習慣——「打開回傳物件看裡面有什麼」。每個 attribute 等下都會用到。

```
response = llm.invoke([{"role": "user", "content": "你好"}])

print("type:", type(response).__name__)
print("text:", response.text())
print("tool_calls:", response.tool_calls)
print("id:", response.id)
print("response_metadata:", response.response_metadata)
print("usage_metadata:", response.usage_metadata)
```

```
type: AIMessage
text: '你好!有什麼我可以幫你的嗎?'
tool_calls: []
id: 'run-xxxxx'
response_metadata: {'finish_reason': 'STOP', ...}
usage_metadata: {'input_tokens': 8, 'output_tokens': 12, ...}
```

.content vs .text()

你以為拿訊息文字就是 `.content` ? **Gemini 2.5** 帶了 **reasoning(thinking)** 機制, `.content` 不再是純字串,而是包了 metadata 的 list。

`response.content`

底層格式 — 給框架用的

```
[{'type': 'text',  
  'text': '你好!...',  
  'extras': {'signature': '...'}}]
```

print 出來一長串 dict,人看了皺眉

`response.text()`

helper method — 給人看的

```
'你好!有什麼我可以幫你的嗎?'
```

純字串,直接可印

習慣:今天 **print** 訊息文字一律用 `.text()`

`.content` 在沒 reasoning 的舊模型(GPT、Claude 早期版)還是字串,但 Gemini 2.5、Claude Sonnet 4.5 等新世代都改成 list 結構—— `.text()` 是 LangChain 統一抽純文字的標準介面

物件的每個 attribute

今天會直接用到的有兩個(`.text()` 、 `.tool_calls`),其他兩個是**進入 production 必看的**——一定要有印象。

`.text()`

訊息文字 — **今天最常用**

用 tool 時可能是空字串(\$8 會看到)

`.tool_calls`

LLM 建議呼叫的工具 — **\$8 開始用**

空 list = 沒呼叫工具,有東西 = 要呼叫

`.usage_metadata`

這次呼叫花多少 token — **production 算錢**

{input_tokens, output_tokens, total_tokens}

`.response_metadata.finish_reason`

STOP = 正常 / MAX_TOKENS = 被截斷

輸出截斷會導致 JSON parse 錯誤等怪 bug

§6.3 多輪對話:messages 是 list

這是 LLM 一個常被誤解的事實:**API 本身沒有「對話狀態」**。每次 invoke 都是獨立的——你不傳歷史,它就不知道。

```
response = llm.invoke([
    {"role": "user", "content": "我叫小明"},
    {"role": "assistant", "content": "你好,小明!"},
    {"role": "user", "content": "我叫什麼名字?"}
])
print(response.text())
# '你叫小明。'
```

LLM 沒有記憶

每次 invoke 要把整段歷史傳進去 — 它才知道剛才聊過什麼

所以 LangGraph 把 messages 包成 list、要 append 不要覆蓋(\$7 會教 reducer)

Debug LLM 最重要的習慣

有不確定就 print 出來看

今天剩下時間,你會看到我每跑完一個 cell
都打開回傳物件 — 這不是表演,是真的工作習慣

§7

MessagesState

對話 agent 的標準寫法

LLM 沒有記憶 → graph 怎麼累積 messages?

LangGraph 的解法是內建 **State + reducer 機制**——不用自己刻。

用 `MessagesState`

LangGraph 內建的 State 類別,專門給對話用——你不用自己定義

內建 `add_messages` reducer

告訴 LangGraph:當你 return 新訊息時,**append** 到舊 **list** 後面,不是覆蓋

接下來看完整實作,一行行讀

完整 chatbot graph

跟 \$4 echo bot 結構一模一樣——差別只在 **chatbot** 節點裡呼叫了 LLM。

```
from langgraph.graph import MessagesState, StateGraph, START, END
from langchain_google_genai import ChatGoogleGenerativeAI

llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0.7)

def chatbot(state: MessagesState) -> dict:
    response = llm.invoke(state["messages"])
    return {"messages": [response]}

builder = StateGraph(MessagesState)
builder.add_node("chatbot", chatbot)
builder.add_edge(START, "chatbot")
builder.add_edge("chatbot", END)

graph = builder.compile()
result = graph.invoke({
    "messages": [{"role": "user", "content": "幫我寫 hello world"}]
})
print(result["messages"][-1].text())
```


MessagesState 內部

打開 LangGraph 原始碼看,MessagesState 其實只有兩行。但這兩行是整個對話 agent 的核心。

```
class MessagesState(TypedDict):  
    messages: Annotated[list, add_messages]
```

§3.3 預告的 Annotated 派上用場了

怎麼讀這行?

messages 是個 list,但 LangGraph 處理時
不要覆蓋,要用 add_messages 合併(append)

記憶口訣:Annotated 第一格是真型別,第二格是給框架看的提示

反例:沒有 add_messages 會怎樣

親眼看到 bug 比聽抽象解釋有效十倍。同樣的 chatbot 邏輯,只把 State 換成沒有 reducer 的版本——bug 馬上出現。

```
class BadState(TypedDict):
    messages: list    # ← 預設行為是「覆蓋」

def bad_chatbot(state: BadState) -> dict:
    response = llm.invoke(state["messages"])
    return {"messages": [response]}    # 覆蓋整個 list

# 跑一次
r1 = bad_graph.invoke({
    "messages": [{"role": "user", "content": "我叫小明"}]
})
print(len(r1["messages"]))
# 1    ← 怪!user 訊息消失了
```

Bug 原因: return {"messages": [response]} 預設覆蓋整個 list

原本「我叫小明」直接消失 → 加上 Annotated[list, add_messages] 就 fix

State 的紀律

這個原則來自函數式程式設計——**不可變(immutable)**資料更易追蹤、易測試。

錯誤寫法

```
def bad_node(state):  
    state["messages"].append(new_msg)  
    return state
```

直接改 state 物件 → 繞過 reducer → 行為難預測

正確寫法

```
def good_node(state):  
    return {"messages": [new_msg]}
```

回新 dict → reducer 處理合併 → 行為可預測

永遠不要直接 mutate state

把新東西包成 dict return 出去,讓 reducer 幫你 merge

§8

Tool Calling

(前半場核心)

bot $\neq$ agent

agent 會

自己決定

要呼叫工具

Tool 與 ReAct loop

這兩個是 agent 工程的核心抽象。理解它們之後,所有 agent 框架(LangGraph、OpenAI Agents SDK、CrewAI...)都是這兩件事的不同包裝。

Tool

你給 LLM 用的「工具」,本質是 **Python function**

例如:查天氣、查資料庫、發 email、執行 SQL、跑 pandas

ReAct loop

LLM 決策 → 執行工具 → 結果回 LLM → 再決策

名稱來自 2022 ReAct paper(Reasoning + Acting),現在是 agent 標準套路

LLM 怎麼決定呼叫 tool?

① 把工具的「使用說明書」附在 LLM 的 API request

② LLM 看問題,決定:文字回答 or 建議呼叫工具

進 LangGraph 之前先看 LLM 回什麼

為什麼要先用「`llm.bind_tools()`」而不是直接放進 LangGraph?——讓你看清楚 LLM 對 tool 的回應長怎樣。LangGraph 後面只是把這個流程自動化。

```
from langchain_core.tools import tool

@tool
def get_weather(city: str) -> str:
    """查詢城市的天氣。輸入城市名,回傳天氣描述。"""
    return f"{city} 今天晴天,25 度"

# 把 tool 綁到 LLM 上
llm_with_tools = llm.bind_tools([get_weather])

response = llm_with_tools.invoke([
    {"role": "user", "content": "台北天氣怎樣?"}
])

print("content:", repr(response.content))
print("tool_calls:", response.tool_calls)
```

注意:這頁特別用 `.content` 不是 `.text()` ——因為要看「LLM 用 tool 時 content 是空的」這個原始信號。下一頁拆解。

LLM 的回應拆解

這就是「LLM 建議呼叫工具」的真實長相——一個 dict, 裡面寫「叫什麼名字的 function」、「參數是什麼」。

```
content: ''      ← 注意是空字串! LLM 不打算用文字回答

tool_calls: [{ 'name': 'get_weather',
                'args': { 'city': '台北' },
                'id': '...',
                'type': 'tool_call' }]
```

- ▶ 用 tool 時, content 通常是空字串(LLM 還沒寫文字)
- ▶ tool_calls 四個 key: name (叫什麼)、args (參數)、id (配對結果)、type (固定)
- ▶ LLM 沒有真的執行 get_weather —— 只是建議
- ▶ 我們要拿這個 dict 自己呼叫對應的 function, 然後把結果包回 ToolMessage 給 LLM

沒有魔法

這就是「LLM 怎麼決定呼叫 tool」的全貌

LangGraph 的 ToolNode 等下會幫我們做：
讀 tool_calls → 呼叫 function → 包回 ToolMessage

§8.3 docstring 是 LLM 唯一的「使用說明書」

這節雖然短,但是 agent 工程的**最大決勝點**。docstring 寫不好,你的 agent 怎樣調都不會聰明。

@tool 把 function 名稱、參數、docstring 包成 JSON schema 餵給 LLM

LLM 完全靠 docstring 決定要不要呼叫這個 tool

docstring 寫不好 = LLM 永遠不呼叫

好的 docstring 要寫:

- • 做什麼
- • 何時該呼叫
- • 參數是什麼意義
- • 回傳什麼

常見錯誤:

- • 只寫一句「查詢天氣」
- • 沒講何時該呼叫
- • 參數沒解釋格式
- • 用中英混雜的怪用語

你的 agent 行為好不好

80%

取決於 tool docstring 寫得好不好

完整 Data Analyst Agent — 工具

注意 docstring 的寫法——「執行任何 groupby/filter/agg 之前必須先呼叫」這種「強指引」是讓 LLM 行為穩定的關鍵。

```
@tool
def inspect_data(path: str) -> str:
    """檢查 CSV 資料品質。執行任何 groupby/filter/agg
    之前**必須**先呼叫這個工具,了解資料的形狀、欄位、
    缺失值情況。

    參數:
        path: CSV 檔案路徑
    回傳:
        字串, 包含 shape、欄位、缺失值、前五筆樣本
    """
    df = pd.read_csv(path)
    return (
        f"Shape: {df.shape}\n"
        f"Columns: {list(df.columns)}\n"
        f"Missing: {df.isna().sum().to_dict()}\n"
        f"Head:\n{df.head().to_string()}"
    )
```

Gemini 對短 docstring 比 Claude 更敏感——詳細寫, LLM 才會穩定呼叫

完整 Data Analyst Agent — run_pandas

這個 tool 接受 LLM 生出的 pandas code,然後跑——強大但危險。下一頁會講為什麼。

```
@tool
def run_pandas(path: str, code: str) -> str:
    """對 CSV 執行 pandas 分析程式碼,例如 groupby、
    filter、agg、sort 等。讀好的 DataFrame 以 `df`
    變數提供,可直接在 code 中使用。

    參數:
        path: CSV 檔案路徑
        code: 一段 pandas expression
    回傳:
        執行結果的字串(超過 2000 字會截斷)
    """
    df = pd.read_csv(path)
    safe_globals = {"df": df, "pd": pd}
    try:
        result = eval(code, safe_globals)
        return str(result)[:2000]
    except Exception as e:
        return f"Error: {e}"
```

eval() 的安全告誡

在你拿這份 code 改成 production 版本之前,停下來看這頁——這個 `eval()` 寫法在 production 是嚴重漏洞。



Production 絕對不能這樣寫

`run_pandas` 用 `eval()` 是為了讓程式碼短到可以講解

LLM 生出來的 code 直接 `eval =` 把程式控制權交給 LLM

LLM 可能被 *prompt injection* 騙去寫 `__import__('os').system('rm -rf /')`, 你的 server 就完了

Production 必須用:

Sandbox 子進程

用 subprocess 跑、限制檔案/網路存取

RestrictedPython

只允許白名單的 Python 操作

e2b / Modal

專門的 code 執行環境,業界主流

完整 Data Analyst Agent — 組裝

這就是所有 **ReAct agent** 的標準骨架——把它記下來,90% 的 agent 場景照這個模式寫就好。

```
from langgraph.prebuilt import ToolNode, tools_condition

tools = [inspect_data, run_pandas]
llm = ChatGoogleGenerativeAI(
    model="gemini-2.5-flash",
    temperature=0.7,
).bind_tools(tools)

def chatbot(state: MessagesState) -> dict:
    return {"messages": [llm.invoke(state["messages"])]}

builder = StateGraph(MessagesState)
builder.add_node("chatbot", chatbot)
builder.add_node("tools", ToolNode(tools))
builder.add_edge(START, "chatbot")
builder.add_conditional_edges("chatbot", tools_condition)
builder.add_edge("tools", "chatbot")
graph = builder.compile()
```

注意:這版故意沒有 **system prompt** — 等下你會看到 agent 「自由發揮」會出什麼問題,後半場再來修

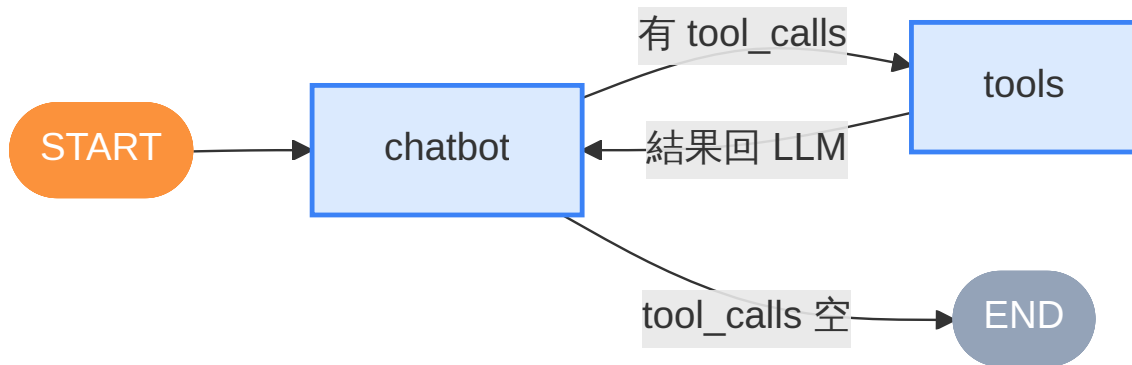
逐行解剖

這個 graph 共用了七個關鍵元件——很多是前幾節已經看過的。能對上每行的功能,就代表你前半場 **follow** 上了。

元件	做什麼	對應
@tool	function → LLM 看得到的工具	\$3.4 \$8.3
bind_tools(tools)	工具說明書附在 LLM 呼叫	\$8.2
chatbot 節點	呼叫 LLM,可能回文字或 tool_calls	\$6
ToolNode(tools)	讀 tool_calls、執行、包結果	\$8.2 我們手動做的事
tools_condition	內建路由 function	有 tool_calls → tools / 沒 → END
add_conditional_edges	條件邊 :依 fn 回傳值動態決定下一站	跟 add_edge 不同
add_edge("tools", "chatbot")	工具跑完回去 LLM	形成循環

本質上 LangGraph 是把「拿 tool_calls → 自己呼叫 → 包 ToolMessage 回去」這串 boilerplate 自動化

ReAct 循環



LLM 決策 → ToolNode 執行 → 結果回 LLM → 再決策

直到 LLM 認為夠了,不再回 tool_calls,走 END

試跑這個 agent 三次

這是整堂課的關鍵時刻——你會親眼看到 LLM agent 的「stochastic 本性」。記下來,後半場要解的就是這個現象。

```
result = graph.invoke({
    "messages": [{ "role": "user",
                    "content": "幫我分析 sales.csv, 找出表現最差的區域"}]
})
print(result["messages"][-1].text())
```

預期看到三次不一致

第一次

可能 East 是最差

第二次

可能 West 是最差

第三次

可能跳過 inspect_data, 結果可能錯

先別急著找原因——這個現象本身就是後半場的主題

把 messages list 整個打開

跟 S6 我們做過的「打開 AIMessage 看 attribute」一樣的習慣——**不確定就 print 出來**。這次打開的是整個 messages list。

```
for i, msg in enumerate(result["messages"]):
    print(f"--- Message {i}: {type(msg).__name__} ---")
    if hasattr(msg, "tool_calls") and msg.tool_calls:
        print("tool_calls:", msg.tool_calls)
    else:
        # 一般文字訊息用 .text() 取出純字串
        text = msg.text() if hasattr(msg, "text") else msg.content
        print("text:", repr(text)[:200])
    print()
```

```
--- Message 0: HumanMessage ---
text: '幫我分析 sales.csv...'

--- Message 1: AIMessage ---
tool_calls: [{'name': 'inspect_data', ...}]

--- Message 2: ToolMessage ---
text: 'Shape: (100, 5)...'

... (繼續疊加)
```

ReAct loop 在 messages list 的真實長相

所謂「agent 自己決定」的真實內容——就是 **messages list** 在那邊一輪一輪疊新訊息。沒有什麼神秘的內部 state machine。

HumanMessage 使用者輸入

AIMessage content 空、有 tool_calls → LLM 決定呼叫工具

ToolMessage 工具執行結果(ToolNode 自動建)

AIMessage content 有東西、沒 tool_calls → LLM 給最終回答

每一輪「LLM 決定 → 執行工具」就在 **messages list** 裡疊兩個訊息
這就是「agent 自己決定下一步」的實質內容 — 不是魔法,是 dict + list

§9

前半場小結

我們做了什麼

前半場 4 個基本功——這四件事組合起來,就是 90% 的 agent 應用。

§4-5

資料水管

State + Node 寫第一個 graph

理解:節點透過 state 傳資料,不直接互相呼叫

§6

打開黑箱

LLM 的 AIMessage 物件解剖

習慣:不確定就 print 出來看

§7

累積對話

MessagesState + add_messages

紀律:不要 mutate state,return 新 dict

§8

ReAct loop

@tool + bind_tools + ToolNode + tools_condition

關鍵:agent 行為穩不穩 = docstring 寫得好不好

LLM agent 的本質特性

記得 S8 試跑那三次嗎?那不是「程式有 bug」,而是 LLM 工程必須面對的**第一個現實**。

剛才講師跑三次同樣問題,結果不一樣

1

East 是最差

正確走完工具流程

2

West 是最差

流程相同但結果不同

3

跳過 inspect_data

流程錯,結果可能錯

這不是 bug、不是寫錯

同一程式、同一資料,每次跑可能走不同路徑、做不同決定

業界所有人都在處理這件事——這就是後半場 *trace + eval* 要解決的

後半場要學什麼

LLM 行為不確定 → 你需要兩套工具:看見它在做什麼、量化它做得多好。這就是後半場全部內容。



Trace

看見 agent 的決策

把「最後一句答案」打開成「每一步發生什麼」——讓你知道 bug 出在哪、好行為長怎樣



Eval

量化 agent 的好壞

把「我覺得 v2 好像比較好」變成「v1 pass 40%、v2 pass 80%」——讓改動有依據

接下來進後半場

§10

後半場開場

前半場看到的問題

寫一個 agent 不難——前半場花一小時就寫完了。但用了之後你會發現兩個問題：

同樣的問題,agent 三次回不一樣

我們現在只看得到最後一句答案,中間發生什麼不知道

想改 prompt 看會不會更好

改完之後不知道是「真的變好」還是「我心理作用」

第一個問題

→ **Trace**

看見 agent 的決策過程

第二個問題

→ **Eval**

用數字量化 agent 的好壞

Harness:給 LLM 套上馬具

****Harness(馬具)****這個詞來自工程實踐——

給 LLM 套上一組「**可控制、可觀測、可評估**」的裝備

讓 agent 行為從「藝術品」變成「工業品」

完整 harness 包含四件事

業界 LLM Ops 公司(LangSmith、Langfuse、Braintrust)賣的就是這四件事的工具。今天不靠平台,用 **LangGraph + 100 行 code** 把核心做完。



Trace

「這次 agent 為什麼這樣決策？」

教(\$11)



Replay

「失敗時能不能從中間試？」

課後練習



Eval

「我改了 prompt, 整體變好還是變壞？」

教(\$12)



Loop

「怎麼把上面接成持續改進的迴圈？」

教(\$13)

後半場我們把前半場寫的 agent 一步步包進這個 harness

§11

Trace

(看見 agent 的決策)

問題:剛才到底發生了什麼?

```
result = graph.invoke({"messages": [...]})  
print(result["messages"][-1].text())
```

我們只看得到最後一句答案

中間 agent 呼叫了哪些 tool、看到什麼結果、為什麼這樣決策——

全部不可見

這就是為什麼前半場跑三次出三種答案我們束手無策

解法:用 stream 取代 invoke

LangGraph 同時提供 `invoke` 跟 `stream` ——一個吃掉中間結果只給你 final state,另一個每個節點跑完都吐出來給你。

```
events = list(graph.stream(  
    {"messages": [{"role": "user",  
        "content": "分析 sales.csv, 找出表現最差的區域"}]},  
    stream_mode="updates",  
))  
  
print(events[0])  
# {'chatbot': {'messages': [AIMessage(content='', tool_calls=[...])]}}
```

stream_mode='updates' — 每個節點跑完後吐出「return 了什麼」

還有 `'values'` (整個 state 快照)、`'messages'` (token-by-token)等模式

events 是 list,每個元素是 {節點名字: 節點 return 的 dict}

invoke 只給你終點;stream 把過程攤開

把 trace 包成漂亮的 print

直接看 `events` 的 raw output 太雜——包成 **helper function** 讓輸出長得像 log,等下一頁就能看出 agent 走了哪些步驟。

```
def pretty_trace(graph, user_message: str):
    """跑 graph,把每個節點的決策漂亮印出來。"""
    inputs = {"messages": [{"role": "user", "content": user_message}]}
    for event in graph.stream(inputs, stream_mode="updates"):
        for node_name, update in event.items():
            print(f"\n[{node_name}]")
            for msg in update.get("messages", []):
                if hasattr(msg, "tool_calls") and msg.tool_calls:
                    for tc in msg.tool_calls:
                        print(f"  → 呼叫工具:{tc['name']}({tc['args']})")
                # 用 .text() 抽純文字(reasoning model 安全)
                text = msg.text() if hasattr(msg, "text") else (msg.content or "")
                if text:
                    print(f"  → 訊息:{text[:500]}")

pretty_trace(graph, "分析 sales.csv,找出表現最差的區域")
```


trace 輸出長相

這就是「agent 走了哪些路」的**實際輸出**。Production 環境會把這串送進 LangSmith / Langfuse,但本質上就是這個樣子。

```
[chatbot]
  → 呼叫工具:inspect_data({'path': 'sales.csv'})

[tools]
  → 訊息:Shape: (100, 5)
      Columns: ['date', 'region', ...]
      Missing: {'region': 8, ...}

[chatbot]
  → 呼叫工具:run_pandas({'path': 'sales.csv', 'code': '...'})

[tools]
  → 訊息:region    revenue
      East      ...

[chatbot]
  → 訊息:East 區表現最差...
```

每個節點的順序、呼叫了什麼工具、看到什麼結果 — 全部看得見

用 trace 解釋「為什麼三次跑不一樣」

還記得 S8 試跑那三次嗎?套上 trace 之後,「神祕變異」變成「具體的決策路徑差異」——可以動手解的問題。

跑法 A(理想)

```
chatbot →  
inspect_data →  
chatbot →  
run_pandas →  
chatbot
```

先 inspect 再分析,結果穩

跑法 B(問題)

```
chatbot →  
run_pandas →  
chatbot
```

↑ 跳過 inspect_data,直接 groupby
沒處理 NaN,結果可能錯

trace 的價值

問題從「神祕」變成「具體可指認的步驟差異」

解法假設:加一句 system prompt 強制要求先 inspect → 但「假設變更有效」需要 S12 量化驗證

動腦不動手

假設你是 agent 開發者
production 上有人回報「agent 偶爾算錯數字」

你會用 **trace** 怎麼 **debug**?

沒有 trace

你只看得到一個錯答案

有 trace

你看得到錯在哪一步

§12

Eval

(用數字說話)

改了 prompt 之後不知道變好變壞

這是 LLM 工程最常見的失敗循環——調整很多次,但不知道哪一次真的有效。Eval 是解這個的工程化方法。

慘痛事實

很多人改 prompt 是靠「感覺好像比較好」

這就是為什麼 LLM 產品難以演進——你沒有客觀基準

更糟:你以為改善了,實際是某些 case 變好、某些 case 變壞,整體可能持平或更差

Eval 是解法

把「agent 應該怎麼做」寫成程式可以檢查的規則

改 prompt → 跑 eval → 看 pass rate 變化

Software engineering 的單元測試思維,搬到 LLM 行為上

Eval case 範例(5 個之 1)

一個 case 包含「問什麼」+「期望什麼」。期望可以是「該呼叫哪些工具」「答案不該出現什麼」「答案該包含什麼」。

```
EVAL_CASES = [  
    {  
        "id": "001_must_inspect_first",  
        "query": "找出表現最差的區域",  
        "must_call_tools_in_order_prefix": [  
            "inspect_data", "run_pandas"  
        ],  
        "answer_must_not_contain": ["NaN", "nan"],  
        "answer_must_contain_one_of": [  
            "East", "West", "North", "South"  
        ],  
    },  
    # ...  
]
```

tool order: 第一個工具一定要是
`inspect_data` (避免跳步)

not contain: 答案不能包含 NaN(代表沒處理乾淨)

contain one of: 答案要明確指出某個區域(不能是 vague 回答)

這些都是「程式可以判斷的規則」——不需要人類來看每個答案

完整 5 個 cases

這 5 個 case 涵蓋不同類型的失敗模式:跳過 inspect、不會處理空結果、漏 inspect、回答不夠明確、不會單純查詢。

```
[
  {"id": "001_must_inspect_first", ...},
  {"id": "002_handle_empty_result",
    "query": "找出 2099 年的資料",
    "answer_must_contain_one_of": ["沒有", "找不到", "無資料"]},
  {"id": "003_inspect_before_summary",
    "query": "幫我做 sales.csv 的整體營收摘要",
    "must_call_tools_in_order_prefix": ["inspect_data"]},
  {"id": "004_top_region",
    "query": "哪個區域表現最好?",
    "must_call_tools_in_order_prefix": ["inspect_data", "run_pandas"]},
  {"id": "005_count_rows",
    "query": "這份資料一共有幾筆?",
    "answer_must_contain_one_of": ["100", "row", "筆"]}
]
```

5 個 case 是個合理起點 — 既能反映 pass rate 變化,又不會花太久跑

每個 case 都對應一種你不希望 agent 做的事

Eval runner

讀法: 跑 graph → 抽 tool_calls 跟 answer → 套 case 裡的規則檢查 → 全 pass 算過。一個 case 一個 case 跑、最後算總體 pass rate。

```
def evaluate_one(graph, case):
    events = list(graph.stream(
        {"messages": [{"role": "user", "content": case["query"]}]}),
        stream_mode="updates",
    ))
    tool_seq = extract_tool_calls(events)
    answer = extract_final_text(events)

    checks = {}
    if "must_call_tools_in_order_prefix" in case:
        prefix = case["must_call_tools_in_order_prefix"]
        checks["tool_order"] = tool_seq[:len(prefix)] == prefix
    if "answer_must_not_contain" in case:
        checks["no_forbidden"] = all(
            kw.lower() not in answer.lower()
            for kw in case["answer_must_not_contain"])
    if "answer_must_contain_one_of" in case:
        checks["has_required"] = any(
            kw.lower() in answer.lower()
            for kw in case["answer_must_contain_one_of"])
    return {"case_id": case["id"], "passed": all(checks.values())}
```

跑第一次 baseline

拿沒加 system prompt 的 graph 跑——這是「**baseline**」:在 LLM 工程裡,你永遠跟「上一版的自己」比,不是跟絕對標準比。

```
report = run_eval_suite(graph, EVAL_CASES)
print(f"Pass rate: {report['pass_rate']:.0%}")
for r in report["results"]:
    status = "PASS" if r["passed"] else "FAIL"
    print(f" {status} {r['case_id']}: {r['checks']}")
```

預期 40%-60%

沒 system prompt 的 agent 行為不穩定——有時跳過 inspect、有時格式不對

每次跑略有不同 — LLM 是 *stochastic* 的; Production 系統的 eval 會跑很多次取平均(N=5 或 N=10)

加 system prompt 看 eval 動

之前 trace 告訴我們「跳過 inspect_data」是 bug 來源——那就用 **system prompt** 強制要求。然後跑同一份 eval, 看 pass rate 動不動。

```
SYSTEM_PROMPT = """你是資料分析助手。
在執行任何 groupby/filter/agg 操作之前,
**必須**先呼叫 inspect_data 檢查資料品質。
如果有缺失值,先在答案中說明你怎麼處理。"""

llm_v2 = ChatGoogleGenerativeAI(
    model="gemini-2.5-flash",
    temperature=0.7,
).bind_tools(tools)

def chatbot_v2(state: MessagesState) -> dict:
    messages = ([{"role": "system", "content": SYSTEM_PROMPT}]
                + list(state["messages"]))
    return {"messages": [llm_v2.invoke(messages)]}

# 重新組 graph_v2、重跑 eval
report_v2 = run_eval_suite(graph_v2, EVAL_CASES)
print(f"Pass rate (v2): {report_v2['pass_rate']:.0%}")
```

注意 line 11-12:把 system prompt 接在使用者訊息前面,讓 LLM 看到約束

量化看到變好

這就是「有 eval 的迭代」跟「沒 eval 的迭代」的差距——同一個 prompt 改動,有 eval 能看到客觀的數字變化。

baseline

40-60%

沒 system prompt

加 system prompt 後

80-100%

強制要求先 inspect

改一行 prompt,量化看到變好

這就是工程化迭代的感覺

沒有 eval 你只能靠「感覺好像比較好」改 prompt;

寫了 eval,你的迭代有方向、有依據、有止損點

重點不是滿分

v2 也不一定 100% — LLM 偶爾還是會不聽話

有客觀基準可以比較
v1 vs v2 是不是真的進步

如果 v2 一樣 40%, 你就知道這個 prompt 沒用, 要試別的
這比「感覺有變好」有用十倍

Eval set 多大才算夠

這是學員進 production 後最常問的問題——「我要多少 case 才夠?」其實思考方式跟你想的**反過來**。

從 1 個開始就比 0 個好

每抓到一個失敗 case 就加一個

有方向比有數量重要

Production LLM 系統的 eval set 通常從幾十個 case 起跳,長到幾百到幾千個

今天你看到的是 5 個——**已經能反映 prompt 改動的影響**

實務心法:不要等 case 寫完才開始用。先寫 1 個 → 跑 → 改 prompt → 發現新 bug → 寫第 2 個 case → 跑 →

eval set 是「長出來」的,不是「計畫出來」的

動腦不動手:第六個 case

如果你要再加一個 eval case
抓 agent 沒抓到的失敗模式

你會想加什麼?

「資料有重複 row 的情境」

「欄位名稱大小寫混淆」

「使用者問題是英文」

「同時 group 兩個維度」

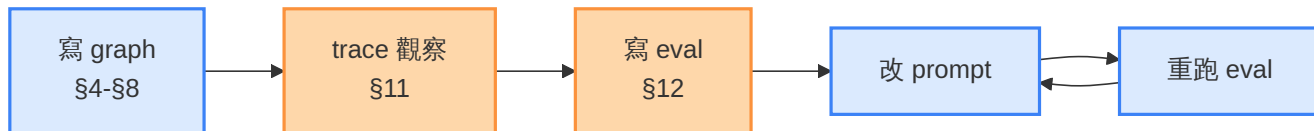
這就是 eval set 怎麼長大的

§13

Harness Loop

把全部串起來

今天你看到的東西已經是**完整 harness 迴圈**的核心。Production LLM 工程就是這個迴圈跑很多輪——每輪改一點、量一次、決定下一步要做什麼。



沒講到的延伸主題：

- • Replay(從 checkpoint 重跑)— 課後練習
- • 框架比較(OpenAI Agents SDK 等)— 附錄
- • Multi-agent(多個 agent 協作)

下一步建議：

- • 把今天的 5 個 case 擴到 10-20 個
- • 接 LangSmith 看 trace 的 production 樣貌
- • 試試把 graph 包成 FastAPI 對外服務

今天最重要的一句話

有 trace + 有 eval =

你的 **agent** 從

「demo 程式」

變成

「可工程化迭代的系統」

§14

Cheatsheet

LangGraph 必踩的 6 個雷

這 6 個是**每個 LangGraph 新手都會踩到的**——把這頁拍下來,寫 code 卡住時翻一下省 30 分鐘 debug。

1 `state["messages"].append(...)` → `return {"messages": [new_msg]}`

2 自訂 State 沒寫 reducer,訊息列表被覆蓋 → `Annotated[list, add_messages]`

3 沒 `compile()` 就 `invoke` → `graph = builder.compile()` 後

4 `@tool docstring` 沒寫「使用時機」 → LLM 不知何時呼叫

5 路由函數裡呼叫 LLM → 路由只讀 `state` 純值

6 Tool 裡用 `eval()` 跑 LLM 給的 code → Production 改 `sandbox / RestrictedPython`

Harness 自檢清單

寫 production agent 之前自問——這 5 點任一個沒打勾,代表你還在 demo 階段,不是 production。

☐ **能 trace 嗎?** 看得到每個節點的決策嗎?

☐ **State 顯式嗎?** 節點之間傳的資料都在 schema 裡?

☐ **有 eval set 嗎?** 至少 5 個 case 涵蓋已知失敗?

☐ **改了東西能量化嗎?** 一行 command 跑 eval 出 pass rate?

☐ **改 graph 有紀錄嗎?** 每次改有對應的 eval report?

沒打勾不算寫完 agent,只算寫了個 demo

§15

收場

兩個跟著你走的東西

LangGraph 會被取代、Gemini 會出新版、API syntax 會改——這些技術細節三年內都會過時。但下面這兩件事會跟著你十年。



State + Node + Tool 的 mental model

agent 不是魔法,是 dict 在節點之間流動 +
LLM 決定下一步呼叫什麼工具



Trace + Eval 的工程紀律

沒有這兩件事,agent 永遠停在 demo 階段;
有了這兩件事,agent 才能可工程化迭代

框架會變,但這兩件事不會變

回去做兩件事

1

把今天的 5 個 **eval case** 自己再加 5 個

涵蓋你能想到的失敗模式

2

挑一個你做過的個人專案

不一定是 agent,任何 LLM 應用都可以

畫出它的「**missing harness**」清單:trace 缺什麼、eval 缺什麼

一個月後

你回來看自己今天看的 code

會覺得簡單

那很正常,那代表你進步了

但你今天看到的
mental model 跟 **harness** 思維會留下

這個比 code 重要

Q&A

謝謝大家